

Behavioral Patterns - Part 3

Memento - Visitor - Null Object - Interpreter

Sub-Chapter 02-07

Memento	Save and restore an object's state (undo/redo)
Visitor	Add new operations to objects without changing their classes
Null Object	Replace null checks with a do-nothing default object
Interpreter	Define a grammar and interpret simple expressions

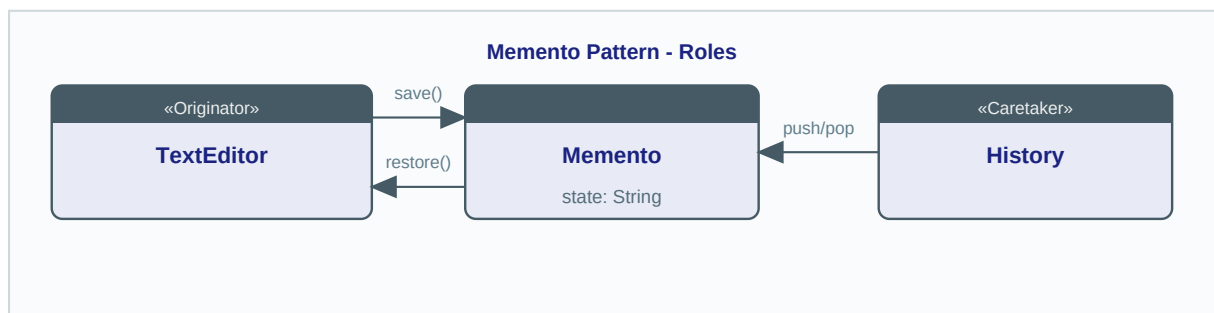
1 Memento

Save and restore an object's state (undo/redo)

The **Memento** pattern lets you capture an object's internal state and restore it later without exposing its private details. Think of it as Ctrl+Z for your objects: the **Originator** creates a snapshot (Memento), a **Caretaker** stores the history, and the Originator restores from any snapshot on demand.

Key Rules

- Originator creates and restores Mementos - it owns the state.
- Caretaker only stores/retrieves Mementos - it never reads the state inside.
- Memento's state field is private - encapsulation is preserved.
- Use a stack (Deque) in the Caretaker to support multi-level undo.



TextEditor (Originator) creates/uses Mementos; History (Caretaker) holds the stack

Java Example - Text editor with undo

```
// Memento -- stores a snapshot
class Memento {
    private String state;
    Memento(String state) { this.state = state; }
    String getState() { return state; }
}

// Originator -- creates and uses mementos
class TextEditor {
    private String text = "";
    void type(String words) { text += words; }
    String getText() { return text; }
    Memento save() { return new Memento(text); }
    void restore(Memento m) { text = m.getState(); }
}

// Caretaker -- holds the history
class History {
    private Deque<Memento> stack = new ArrayDeque<>();
    void push(Memento m) { stack.push(m); }
    Memento pop() { return stack.isEmpty() ? null : stack.pop(); }
}

// Usage
TextEditor editor = new TextEditor();
History history = new History();
editor.type("Hello"); history.push(editor.save());
editor.type(" World"); history.push(editor.save());
editor.type("!!!");
System.out.println(editor.getText()); // Hello World!!!
editor.restore(history.pop());
System.out.println(editor.getText()); // Hello World
editor.restore(history.pop());
System.out.println(editor.getText()); // Hello
```

Q1. What is the role of the Memento class?

It stores a snapshot of the Originator's internal state at a point in time. Its fields are private so only the Originator can access them.

Q2. Why does the Caretaker not read the Memento's contents?

Because the state is implementation-specific to the Originator. Keeping it hidden enforces encapsulation and single responsibility.

Q3. How does undo work with a stack-based Caretaker?

Each save pushes a Memento onto the stack. Each undo pops the last Memento and passes it to restore(), reverting to the previous state.

Q4. What happens if you call pop() on an empty history?

The guard 'stack.isEmpty() ? null : stack.pop()' returns null. The caller should check for null before calling restore().

Q5. Name two real-world uses of the Memento pattern.

Text editor undo/redo, and game save-state systems where the player can reload a saved checkpoint.

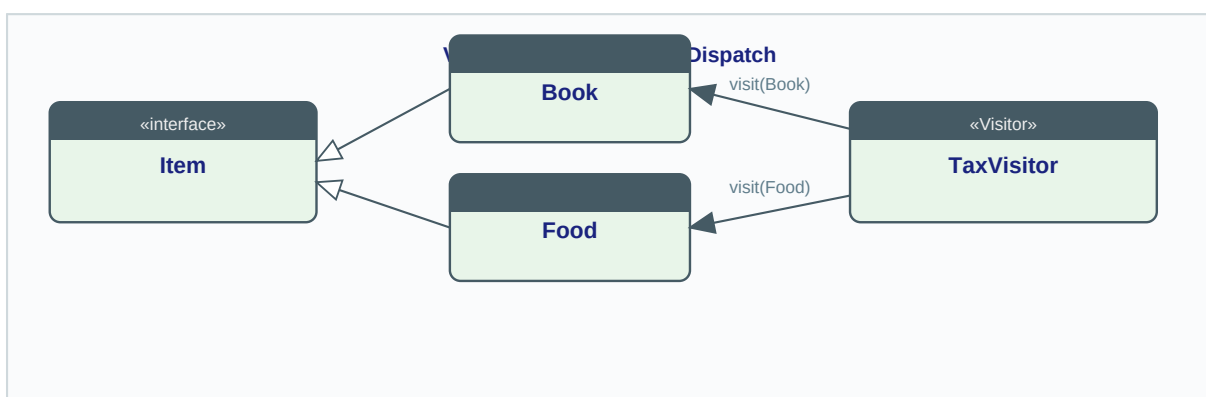
2 Visitor

Add new operations to objects without changing their classes

The **Visitor** pattern separates an algorithm from the objects it operates on. Instead of adding tax-calculation logic inside Book or Food, you create a **TaxVisitor**. Each element calls **accept(visitor)**, which dispatches to the correct **visit()** overload - a technique called *double dispatch*.

Key Rules

- The Element interface declares accept(Visitor) - all concrete elements implement it.
- The Visitor interface has one visit() overload per concrete element type.
- Double dispatch: accept() calls visitor.visit(this), selecting the right overload.
- To add a new operation, add a new Visitor class - no existing element code changes.
- Adding a new element type requires updating all Visitor implementations.



Book and Food implement Item; TaxVisitor visits each with a different overload

Java Example - Shopping cart tax calculator

```
interface Visitor { void visit(Book b); void visit(Food f); }
interface Item { void accept(Visitor v); }

class Book implements Item {
    double price;
    Book(double price) { this.price = price; }
    public void accept(Visitor v) { v.visit(this); }
}
class Food implements Item {
    double price;
    Food(double price) { this.price = price; }
    public void accept(Visitor v) { v.visit(this); }
}
class TaxVisitor implements Visitor {
    public void visit(Book b) {
        System.out.println("Book tax (0%): Rs " + (b.price * 0));
    }
    public void visit(Food f) {
        System.out.println("Food tax (5%): Rs " + (f.price * 0.05));
    }
}

// Usage
List<Item> cart = List.of(new Book(200), new Food(100));
Visitor tax = new TaxVisitor();
for (Item item : cart) item.accept(tax);
// Book tax (0%): Rs 0.0
// Food tax (5%): Rs 5.0
```

Q1. What problem does Visitor solve?

It lets you add new operations to a class hierarchy without modifying those classes, keeping the Open/Closed Principle intact.

Q2. What is double dispatch?

First dispatch: `item.accept(visitor)` selects the concrete Item subclass. Second dispatch: `visitor.visit(this)` selects the correct overload for that subclass.

Q3. When should you avoid Visitor?

When the element hierarchy changes often. Every new element type forces every Visitor implementation to add a new `visit()` method.

Q4. How would you add a DiscountVisitor without touching Book or Food?

Create a new class `DiscountVisitor` implementing `Visitor` with its own `visit(Book)` and `visit(Food)` logic. `Book` and `Food` are untouched.

Q5. What is the role of `accept()` in each element?

It calls `visitor.visit(this)`, passing the concrete type. This is what enables the second dispatch to the right overload.

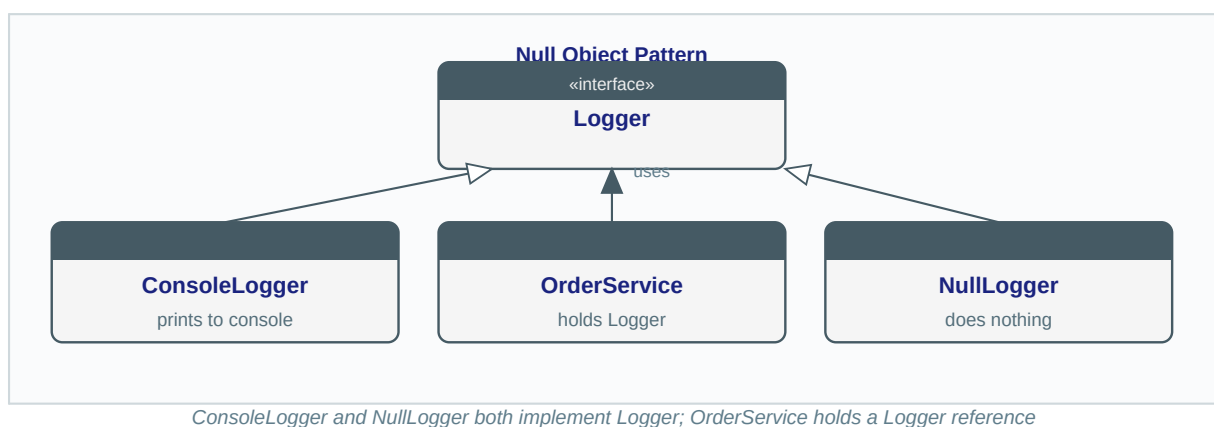
3 Null Object

Replace null checks with a do-nothing default object

The **Null Object** pattern eliminates defensive null checks by providing a do-nothing implementation of an interface. Instead of passing null and scattering if (logger != null) guards, you inject a **NullLogger** that silently ignores every call.

Key Rules

- Define a common interface (Logger) that both real and null objects implement.
- The Null Object's methods do nothing (or return safe defaults like 0 / empty string).
- The client (OrderService) never checks for null - it just calls the interface.
- Inject real or null objects at construction time (dependency injection).
- Avoids NullPointerException and makes code cleaner and easier to test.



Java Example - Pluggable logger

```
interface Logger {
    void log(String message);
}

class ConsoleLogger implements Logger {
    public void log(String message) {
        System.out.println("LOG: " + message);
    }
}

// Null Object -- does nothing, but safe to call
class NullLogger implements Logger {
    public void log(String message) { /* do nothing */ }
}

class OrderService {
    private Logger logger;
    OrderService(Logger logger) { this.logger = logger; }
    void placeOrder(String item) {
        logger.log("Placing order for: " + item);
        System.out.println("Order placed: " + item);
    }
}

// Usage
OrderService svc1 = new OrderService(new ConsoleLogger());
svc1.placeOrder("iPhone"); // LOG: Placing order for: iPhone
                          // Order placed: iPhone

OrderService svc2 = new OrderService(new NullLogger());
svc2.placeOrder("MacBook"); // Order placed: MacBook (no log -- silent)
```

Q1. What specific problem does Null Object solve?

It removes repeated null checks like 'if (logger != null) logger.log(...)' by guaranteeing the reference is always a valid object.

Q2. How is Null Object different from passing null?

Passing null causes NullPointerException if the client forgets a null check. A Null Object is always safe to call - it just does nothing.

Q3. Is NullLogger a subclass or an implementation?

An implementation. It implements the Logger interface. This keeps things flexible and consistent with polymorphism.

Q4. When would you use NullLogger in tests?

In unit tests for OrderService where you don't care about logging output. NullLogger keeps test output clean without mocking.

Q5. What are safe return defaults for non-void Null Object methods?

Return 0 for numbers, empty string for text, empty list for collections, and false for booleans - values that cause no side effects.

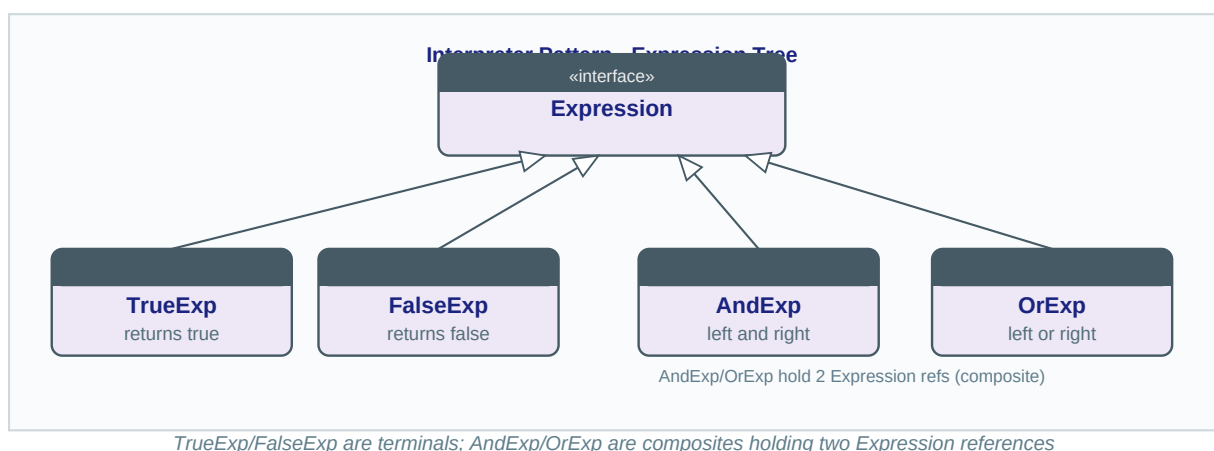
4 Interpreter

Define a grammar and interpret simple expressions

The **Interpreter** pattern defines a grammar for a simple language and provides an interpreter to evaluate sentences in that language. Each grammar rule becomes a class. Composite expressions (AndExp, OrExp) hold sub-expressions, forming a tree that is evaluated recursively.

Key Rules

- Terminal expressions (TrueExp, FalseExp) return a value directly.
- Non-terminal expressions (AndExp, OrExp) compose two sub-expressions.
- All expressions implement the same interface: boolean interpret().
- The pattern shines for simple grammars; use a real parser for complex languages.
- Each grammar rule maps to exactly one class - easy to extend with new rules.



Java Example - Boolean expression interpreter

```
interface Expression { boolean interpret(); }

class TrueExp implements Expression { public boolean interpret() { return true; } }
class FalseExp implements Expression { public boolean interpret() { return false; } }

class AndExp implements Expression {
    private Expression left, right;
    AndExp(Expression l, Expression r) { left=l; right=r; }
    public boolean interpret() { return left.interpret() && right.interpret(); }
}
class OrExp implements Expression {
    private Expression left, right;
    OrExp(Expression l, Expression r) { left=l; right=r; }
    public boolean interpret() { return left.interpret() || right.interpret(); }
}

// Usage
// true AND false = false
Expression exp1 = new AndExp(new TrueExp(), new FalseExp());
System.out.println(exp1.interpret()); // false

// true OR false = true
Expression exp2 = new OrExp(new TrueExp(), new FalseExp());
System.out.println(exp2.interpret()); // true

// (true OR false) AND true = true
Expression exp3 = new AndExp(new OrExp(new TrueExp(), new FalseExp()), new TrueExp());
```



```
System.out.println(exp3.interpret()); // true
```

Q1. What is the terminal expression in this example?

TrueExp and FalseExp. They have no children and return a value directly - they are the leaves of the expression tree.

Q2. What makes AndExp and OrExp non-terminal?

They hold references to two sub-expressions (left and right) and delegate to them by calling interpret() recursively.

Q3. How would you add a NotExp without changing existing classes?

Create class NotExp implementing Expression, holding one Expression. Its interpret() returns !child.interpret(). No other class changes.

Q4. Why is Interpreter not recommended for complex grammars?

For complex grammars the number of classes grows quickly and performance degrades. Tools like ANTLR or PEG parsers are better suited.

Q5. Name a real-world use of the Interpreter pattern.

SQL query engines, regular expression matchers, template engines, and scripting language evaluators all use variants of this pattern.

Cheat Sheet

Behavioral Patterns Part 3 - Quick Reference

Pattern	Intent	Key Roles	Real Use Case	Spot Violation
Memento	Save and restore an object's internal state	Originator, Memento, Caretaker	Text editor undo, game save-states	Caretaker reads Memento fields directly (breaks encapsulation)
Visitor	Add operations to objects without modifying their classes	Visitor interface, Element interface, ConcreteVisitor	Tax calculator, report generator on object graphs	Adding if/instanceof inside visitor instead of overloaded visit()
Null Object	Replace null with a safe do-nothing object	Interface, RealObject, NullObject, Client	Silent logger in tests, default no-op event handler	Null Object methods throw exceptions or have side effects
Interpreter	Define a grammar and evaluate expressions	Expression interface, TerminalExp, NonTerminalExp	Boolean rule engines, SQL WHERE clause parsers	Putting all grammar logic in one God-class instead of one class per rule